AI-Enhanced Smart Gesture Communication Glove for Assistive Communication

Lingesh P., Derin Gino G., Akash S., and Meenakshi N.

Department of Artificial Intelligence and Machine Learning, School of Engineering
Vels Institute of Science, Technology & Advanced Studies (VISTAS), Chennai, Tamil Nadu, India

Faculty Mentor: Dr. N. Meenakshi

***Abstract*—**Communication between people who use sign language and people unfamiliar with it can be difficult when a shared interpreter or communication medium is unavailable. This paper presents an AI-enhanced wearable glove prototype intended to support assistive communication by recognising a predefined set of hand gestures and presenting the associated word as text. The prototype combines five flex sensors to capture finger bending with an MPU6050 inertial measurement unit (IMU) that provides three-axis accelerometer and gyroscope readings. An ESP32 microcontroller acquires the sensor signals and coordinates the recognition workflow. A supervised Random Forest classifier is used in the gesture-to-text pipeline, with predicted classes mapped to configured word labels for display. The design is camera-independent and intended for isolated, predefined gestures rather than unrestricted sign-language translation. The project team reports integration of the sensing, classification, and text-output pipeline; however, verified numerical metrics such as test accuracy, precision, recall, F1-score, and inference latency were not available for this paper. Current diagnostic logs from a subsequent firmware session reported a missing model header and an OLED detection issue, so end-to-end operation should be revalidated on the exact build submitted with this paper. The paper documents the system architecture, implementation approach, evaluation requirements, limitations, and directions for further work.

***Index Terms*—**Assistive communication, gesture recognition, flex sensors, MPU6050, ESP32, Random Forest, wearable computing, Indian Sign Language.

I. INTRODUCTION

Communication is a basic requirement for education, healthcare, employment, public services, and daily social interaction. People who communicate through sign language may encounter barriers when a conversation partner does not understand the signs being used. Access to a human interpreter is not always immediate, and written communication may not be convenient in every situation. Assistive technologies that provide an additional channel between a signer and a non-signing conversation partner can therefore be useful when designed around the needs and preferences of the people using them.

Hand gestures are expressed through combinations of finger configuration, hand orientation, and movement. A wearable glove can capture some of these physical signals using sensors attached to the hand. Unlike a camera-based approach, a sensor glove does not require an unobstructed camera view or a particular lighting condition. However, it introduces its own challenges, including sensor calibration, repeatable placement, mechanical comfort, electrical noise, and differences between users.

This project explores a sensor-based glove that combines five flex sensors with an MPU6050 inertial measurement unit (IMU). The flex sensors measure changes associated with bending of the thumb and four fingers, while the IMU contributes acceleration and angular-velocity readings. An ESP32 microcontroller coordinates sensor acquisition and the gesture-recognition workflow. A Random Forest classifier is used in the intended gesture-to-text pipeline, in which a predicted class is mapped to a predefined word label.

The objective is to build a portable prototype for recognising a limited vocabulary of predefined hand gestures and presenting the associated text. The system is not intended to claim unrestricted translation of Indian Sign Language (ISL). Full sign-language interpretation may require facial expression, body posture, spatial grammar, context, and transitions between signs, none of which are fully represented by the glove's current sensor set.

A. Contributions

The work brings together (1) five-channel finger-bending sensing, (2) inertial sensing from an accelerometer and gyroscope, (3) a supervised Random Forest classification approach for structured sensor data, and (4) a local text-output path intended for embedded use. It also identifies practical implementation and evaluation requirements that must be met before performance claims can be made.

B. Paper Organisation

Section II reviews relevant technical approaches and the background needed to understand the design. Section III describes the system architecture and hardware. Section IV explains data preparation, model training, and embedded integration. Section V documents implementation status and proposes a reproducible evaluation. Section VI discusses limitations and future work, followed by the conclusion and references.

II. RELATED WORK AND TECHNICAL BACKGROUND

Gesture-recognition systems are commonly developed using vision-based sensing, wearable sensors, or combinations of both. Vision-based methods can capture hand shape and contextual information from images or video. Their practical performance can be influenced by lighting, background clutter, camera angle, occlusion, and the distance between the hand and camera. These issues do not make vision-based recognition unsuitable, but they can complicate deployment in environments where camera placement or privacy is a concern.

Wearable approaches measure hand-related signals directly through sensors attached to a glove or another body-worn device. Flex sensors are useful for detecting changes in finger curvature. Their readings are affected by the sensor's physical placement, bend radius, mounting tension, voltage-divider design, and individual sensor characteristics. Calibration is therefore important: raw analog-to-digital converter (ADC) readings should not automatically be interpreted as universal bend angles or as identical values across different gloves.

Inertial measurement units provide complementary motion information. An accelerometer measures specific force along three axes, including the effect of gravity, while a gyroscope measures angular velocity along three axes. These channels can help distinguish gestures with similar finger configurations but different hand orientations or movements. Conversely, inertial data alone may be insufficient to distinguish static finger shapes. Combining flex and IMU signals is thus a plausible multimodal design choice, but its benefit must be tested against flex-only and IMU-only baselines.

Machine-learning models can map structured sensor features to gesture labels. Random Forest, introduced by Breiman [1], combines predictions from multiple decision trees. It can represent nonlinear decision boundaries and can work with tabular data without requiring image processing. Its practical performance still depends on training-data coverage, label quality, class balance, hyperparameter selection, and the independence of the evaluation set.

The system also depends on reliable embedded integration. The ESP32 family provides microcontroller capabilities suitable for sensor acquisition and local processing [2]. The MPU6050 communicates through a digital interface and provides accelerometer and gyroscope measurements [3]. If the fitted display uses an SSD1306 controller, its command set and electrical requirements should be checked against the relevant controller documentation [4]. The exact flex-sensor model and its recommended electrical circuit should likewise be confirmed from the supplier documentation [5].

A. Design Rationale

The design favours a camera-independent, low-complexity sensing path that can be worn and operated locally. This may reduce dependence on external computing and continuous network connectivity. The trade-off is that the glove measures only the signals available from its installed sensors, and a finite labelled dataset must represent each gesture that the classifier is expected to recognise.

B. Research Gap Addressed by the Prototype

The prototype addresses an engineering integration problem: bringing finger-bending measurements and inertial readings into a consistent embedded classification workflow. The contribution at this stage is a design and implementation pathway, not a claim that sensor fusion has already been proven superior or that the system provides general-purpose sign-language translation. Those claims would require controlled comparative experiments and broader user testing.

III. SYSTEM ARCHITECTURE

The system can be described as four connected stages: sensing, acquisition, classification, and output. The sensing stage produces five flex channels and six inertial channels. The acquisition stage reads those channels through the ESP32 and assembles the feature vector. The classification stage applies a trained Random Forest model to estimate the gesture class. The output stage maps that class to a predefined word or phrase and updates the display when the display hardware is available and functioning.

A. Hardware Components

The main components are an ESP32 development board, five flex sensors, appropriate voltage-divider circuits, an MPU6050 IMU, a 0.96-inch OLED module, a glove base, wiring, and the power hardware used in the physical prototype. Each flex sensor is positioned to respond to bending of one digit: thumb, index, middle, ring, and little finger. Sensor placement should allow normal hand movement while limiting sharp bends or repeated mechanical strain at the sensor connections.

The documented ESP32 pin mapping is GPIO 33 for thumb, GPIO 32 for index, GPIO 35 for middle, GPIO 34 for ring, and GPIO 36 for little finger. The documented I²C mapping for the MPU6050 is SDA on GPIO 21 and SCL on GPIO 22. These are project-specific assignments, not universal requirements for every ESP32 board. Before submission, the wiring diagram and final source code should be checked to ensure that the physical connections match these assignments.

Flex sensors are commonly read through an analog voltage-divider circuit. The ADC reading depends on the sensor resistance, fixed resistor value, supply voltage, ADC attenuation and resolution, and the mechanical bend. The divider should be designed so that the voltage presented to the ESP32 remains within the board's permitted input range. A stable common ground and secure connectors are important for repeatable readings. The precise resistor values and power arrangements should be documented from the actual build rather than assumed.

The MPU6050 provides three accelerometer channels and three gyroscope channels. Its readings may be returned as raw integer counts or converted into physical units using the selected full-scale ranges and sensitivity factors. The paper's dataset schema treats these as raw readings unless the firmware explicitly converts them. Raw counts should not be labelled as metres per second squared or degrees per second unless a correct conversion has been applied.

B. Embedded Processing

The ESP32 acts as the interface between sensors and the gesture classifier. A stable acquisition cycle should read all flex channels and all six IMU channels, check for communication errors where possible, and create a feature vector in the same order expected by the training pipeline. The code should distinguish a valid prediction from a diagnostic state in which the model has not loaded. It should not display a gesture label when the classifier is unavailable.

C. Text Output and Diagnostics

The intended user-facing output is the word associated with the recognised gesture on the OLED. During development, serial logging and a local Wi-Fi access-point dashboard have also been used to inspect sensor values and system status. The dashboard is a diagnostic tool rather than proof of recognition accuracy. When a display or model fails to initialise, the firmware should report the failure clearly and avoid presenting stale or misleading output.

D. Functional Flow

The end-to-end sequence is: initialise the sensors and output device; verify device status; load the trained model; acquire sensor measurements; arrange the features; run classification; map the predicted class to a word; and update the output. A robust implementation should handle failed sensor reads, unavailable model files, unknown class identifiers, and display errors without crashing or silently generating incorrect words.

E. Calibration and Signal Quality

Calibration should be performed after the sensors have been mounted on the glove, because mounting tension and finger alignment affect the measured values. A practical procedure records a relaxed, open-hand baseline and several repeatable bent postures for each digit. The objective is not to force every sensor to share the same numeric range, but to identify a stable response range for each physical channel. If the firmware uses raw ADC values, those values should be retained consistently; if normalisation is introduced, the calibration parameters must be stored and applied identically during training and inference.

Signal quality checks should look for abrupt jumps, stuck readings, disconnected channels, and values that remain fixed while the corresponding finger moves. Flex-sensor readings should be checked individually before the complete model is trained. IMU readings should also be checked with the device held still and then moved slowly through known orientations. Such checks can reveal swapped axes, incorrect units, poor ground connections, and inconsistent sampling before these errors become embedded in the dataset.

F. Power and Wearability Considerations

A wearable prototype should be evaluated not only for classification but also for practical use. Sensor wires should be routed to reduce snagging and should not prevent normal bending. The microcontroller and battery should be positioned so that they do not create excessive pressure on the hand. Electrical connections need strain relief, and the power circuit should be checked for safe voltage levels and stable operation. The paper does not claim a measured battery runtime because a verified runtime test was not provided.

The current project uses a local processing approach, which can reduce dependence on continuous internet connectivity for inference. The wireless dashboard used during development is intended for local diagnostics. Its presence does not mean that gesture records are uploaded to a cloud service. If data logging or remote features are added later, the design should document what is transmitted, how consent is obtained, and how sensitive usage records are protected.

IV. METHODOLOGY

A. Gesture Vocabulary and Labelling

A supervised classifier learns a mapping from input measurements to labels, so each recorded sample must have a reliable ground-truth label. For a single-finger-closed experiment, the target label should identify which finger was intentionally bent, while the remaining fingers are held in the specified open posture. The collection protocol should define how the hand is positioned, whether the wrist is neutral, how long a reading is held, and whether the sample is taken during movement or after the posture stabilises.

A useful initial vocabulary can include the fingers-open baseline and isolated thumb-, index-, middle-, ring-, and little-finger-closed postures. More complex signs should be added only when their definitions and sensor patterns can be captured consistently. A posture label must not be confused with a complete ISL sign unless the gesture has been selected and validated as an appropriate representation of that sign.

B. Sensor Data Collection

Each record should contain five flex values and six IMU values, together with the class label and preferably metadata such as session identifier, participant identifier, trial number, and timestamp. These metadata support quality checks and evaluation, even if they are not model inputs. If metadata cannot be recorded by the current firmware, the collection process should document session boundaries separately and avoid implying that the original serial log contains timestamps.

The project has already parsed baseline fingers-open readings and a thumb-closed batch into spreadsheets. These files are useful for verifying the data pipeline and beginning collection, but they are not sufficient on their own to establish robust multi-class performance. Separate recordings are needed for each additional class. Synthetic values created by copying or mathematically modifying another finger's readings must not be mixed with measured training data unless explicitly used in a separately marked simulation study.

Before model training, inspect each channel for missing values, unexpected constants, clipping, disconnected sensors, and out-of-range readings. Plotting distributions by label can help identify whether the labels have distinguishable signal patterns. If the same physical sensor channel is consistently assigned to a different finger due to a wiring or naming error, the model may learn an incorrect mapping even when the code runs without exceptions.

C. Feature Vector

The intended feature vector contains eleven values: flex-thumb, flex-index, flex-middle, flex-ring, flex-little, accelerometer X, accelerometer Y, accelerometer Z, gyroscope X, gyroscope Y, and gyroscope Z. The exact order is critical. Training code and embedded inference code must use the same order, numeric representation, units, and preprocessing. If filtering, smoothing, scaling, or window-based features are introduced later, the same operations must be applied during training and inference.

D. Random Forest Training

The dataset is divided into training and evaluation partitions before model fitting. A Random Forest trains multiple decision trees using sampled data and features, then combines their outputs to produce a class prediction. The model should be trained only on examples from the training partition. Hyperparameter choices, including tree count and depth constraints, should be selected using a validation process rather than by repeatedly checking the final test set.

For sensor streams collected as consecutive serial samples, a random row-wise split can exaggerate performance because adjacent samples may be almost identical and appear in both partitions. A stronger evaluation holds out entire recording sessions, and a participant-independent evaluation is preferable when the intended use includes people not represented in training. If only one participant is available, the paper should state that limitation clearly.

E. Model Export and Embedded Mapping

After training and evaluation, the classifier must be exported in a representation compatible with the ESP32 firmware. The project's embedded interface expects a generated model header containing tree offsets, node counts, child-node indices, feature indices, thresholds, leaf votes, and class names. The generated header must correspond to the same model and class ordering used by the training script. A missing or mismatched header means that predictions cannot be considered valid.

The class-to-word mapping should be explicit and tested for every class. The system should handle an invalid class index and an uninitialised model safely. A useful validation test compares predictions from the exported embedded model with predictions from the original training environment for the same fixed test vectors. Differences may indicate export errors, data-type differences, threshold handling problems, or inconsistent feature order.

F. Text Mapping

A predicted class is converted to a readable word using a predefined mapping table. For instance, a label name may be associated with a specific output word only after the team has defined and tested that association. A generic finger-closed posture should not be described as an ISL word or phrase without a defensible linguistic mapping. The output vocabulary and its intended meaning should be documented and agreed with the users for whom the system is designed.

G. Training-Data Quality Controls

Each label should be assigned at the time of capture or through a verified annotation process. A label should not be inferred solely from the sensor values, because that would make the dataset circular and could reinforce an incorrect assumption. When collecting a single-finger-closed posture, the operator should confirm that the intended finger is bent and the remaining fingers are in the defined reference position. If the posture is ambiguous, the sample should be discarded or separately marked for review rather than assigned a confident label.

Class balance is another important consideration. If one gesture has many more examples than another, a classifier may favour the frequent class. The team should report the number of valid samples per class and consider collecting additional examples for underrepresented classes. Any balancing method must be applied only to the training partition; evaluation data should reflect the intended test conditions rather than an artificially altered distribution.

H. Reproducibility and Version Control

A reproducible model release should retain the training script, dataset version, class-name list, model parameters, and export procedure. The firmware and generated model header should be treated as a matched pair. A change to class ordering, feature order, sensor calibration, or numeric conversion should trigger a new validation run. The paper should identify the firmware version and model version used for final tests if those identifiers are available.

For embedded comparison, a fixed set of recorded sensor vectors can be replayed through the software model and the embedded implementation. The predicted class should match for each vector, subject to any documented differences in numeric precision. This test isolates export and integration problems from the natural variability of a person performing a gesture. It is particularly useful when the firmware reports that a model header is missing or fails to initialise.

I. Failure-State Handling

The system should distinguish between a valid gesture prediction and a failure or waiting state. If the model cannot be loaded, the output should report that classification is unavailable. If the IMU is disconnected, the firmware should not silently substitute stale readings unless that fallback is explicitly designed and validated. Similarly, an OLED initialisation failure should be visible in diagnostics, while the rest of the system should remain safe and debuggable. These behaviours reduce the risk that users interpret an old word on the display as a new, correctly recognised gesture.

V. IMPLEMENTATION STATUS AND EXPERIMENTAL EVALUATION

The project team reports implementation of the glove sensing setup and a Random Forest-based gesture-to-text pipeline. The documented hardware includes five flex sensors and an MPU6050 connected to an ESP32. A text display is part of the intended system, and voice output is outside the current completed scope. The serial data collected to date have been organised into spreadsheets for baseline and thumb-closed postures.

The available project materials do not provide verified numerical results for independent test accuracy, per-class precision, recall, F1-score, confusion matrix, inference latency, power consumption, or cross-user generalisation. Accordingly, this paper does not assign numerical values to these measures. Reporting fabricated or inferred scores would make the evaluation irreproducible and could misrepresent the maturity of the prototype.

A later diagnostic serial session reported that the MPU6050 was detected, but the OLED was not found and the model header was missing. These messages indicate that the exact firmware session did not verify the full prediction-and-display path. They should be treated as unresolved integration issues for that session. The final build should be retested before the paper is submitted as a report of fully working end-to-end behaviour.

A. Verification Checklist

The evaluation should begin with unit-level checks. First, confirm that each flex sensor changes in a repeatable direction when its assigned finger bends and returns toward its baseline when released. Second, confirm that the MPU6050 provides plausible readings and that the axis order is stable. Third, verify that all eleven features are assembled in the intended order. Fourth, verify that the model header loads and that a fixed set of test vectors produces the same class in the training environment and on the embedded target. Fifth, verify that every predicted class maps to the correct word and that the OLED updates reliably.

After these checks, conduct repeated end-to-end trials for every gesture. Use a consistent written protocol and record expected label, predicted label, output displayed, and any error. Trials should include repeated attempts across different sessions. If testing with multiple users is not possible, the paper should explicitly limit its conclusions to the participant and sessions tested.

B. Quantitative Measures

For a labelled test set, overall accuracy is the number of correct predictions divided by the total number of evaluated predictions. Per-class precision measures the proportion of predictions for a class that are correct; recall measures the proportion of true examples of that class that are detected; and the F1-score is the harmonic mean of precision and recall. These per-class measures are important when the dataset is imbalanced or when some gestures are easier to recognise than others.

A confusion matrix should be reported to show which classes are mistaken for one another. Latency should be measured with a defined start and end point, such as from the completion of sensor acquisition to the availability of a predicted label. If response time is measured through display refresh, that boundary should be stated separately. Report the number of samples per class, collection sessions, participants, train/validation/test split, and any exclusions alongside the results.

C. Qualitative Evaluation

Qualitative observations can complement numerical metrics. Useful observations include whether the glove remains comfortable during repeated use, whether the sensors restrict natural finger movement, whether values drift over time, whether the device requires recalibration, and whether users can understand the displayed words. These observations should be collected systematically rather than added from memory after the tests. User studies involving people with disabilities require appropriate consent and careful attention to accessibility and privacy.

D. Results Reporting at the Current Stage

At the time of preparing this version, the verified evidence supports describing the hardware configuration, data-recording workflow, and planned or implemented model pipeline, but it does not support numerical performance claims. The current partial dataset should be described as an ongoing data-collection effort. The complete results section should be updated only after the final model and display path have been tested with a documented test set.

E. Proposed Evaluation Table

For transparent reporting, the final test record should include the following fields: gesture class, number of test trials, correct predictions, incorrect predictions, precision, recall, F1-score, and any observed hardware failures. The table should be populated only from actual test runs. If a class has not yet been tested, it should be marked as not evaluated rather than assigned a zero or an estimated score. If a metric cannot be calculated because no test examples or predictions exist for a class, the paper should explain that limitation.

A separate hardware-verification table can record whether each subsystem passed its test: five flex channels, MPU6050 communication, model loading, class-to-word mapping, OLED initialisation, and end-to-end recognition. The test date, firmware version, model header version, and test operator can be recorded in the lab notes. This provides traceability between a paper's claims and the exact prototype that was demonstrated.

F. Threats to Validity

Several factors may affect the validity of a future evaluation. A small sample set may not represent natural gesture variability. Repeated rows from one recording session may be correlated. Sensor placement can change between trials, and the person collecting the data may unintentionally perform each gesture in a highly consistent way that does not generalise to other users. These risks should be addressed through repeated sessions, clearly defined capture instructions, and held-out testing.

Implementation-specific failures also affect validity. A high score from a desktop model does not prove that the exported embedded model is loading correctly or that the display is showing the same prediction. For this reason, both offline model evaluation and end-to-end hardware tests are required. Claims should state clearly which level was tested: sensor acquisition, offline classification, embedded inference, or complete user-facing operation.

VI. DISCUSSION

The use of both flex and inertial sensors is intended to represent complementary aspects of a hand gesture. Flex measurements capture finger articulation, while the IMU provides information about orientation and movement. This combination could be useful when two gestures share a similar finger posture but differ in motion. However, a multimodal design also increases the number of channels that must be calibrated, synchronised, and maintained. A comparative study is needed to establish whether the additional IMU channels improve classification enough to justify that complexity.

The choice of Random Forest is practical for a relatively small tabular feature set and offers a model structure that can be exported to embedded code. Nevertheless, the model cannot compensate for missing or inconsistent labels. If a class contains only a few recordings, the classifier may overfit those examples. If the same session contributes nearly identical rows to both training and test partitions, the reported score may overstate real-world performance. Data quality and evaluation design are therefore as important as algorithm selection.

The intended text output makes the device more immediately interpretable than raw sensor values, but the semantic mapping requires care. A physical posture is not necessarily equivalent to a complete word in a natural sign language. The vocabulary should be described precisely, and future expansion should involve appropriate sign-language users or experts. The system should be positioned as an assistive communication aid for selected gestures, not a replacement for interpreters or a complete language translation system.

Reliability is also a system-level property. A correct classifier prediction is not useful if the sensor is disconnected, the model file fails to load, the class mapping is inconsistent, or the display remains blank. Status checks, clear error states, and end-to-end tests are required to prevent a failed subsystem from being mistaken for a valid output. The recent diagnostic messages make this distinction particularly relevant to the present prototype.

A. Usability and Accessibility

An assistive device should be evaluated in terms of whether its output is understandable, timely, and useful for the intended user. The display text should be legible, and the mapping between gesture and word should be predictable. The glove should avoid requiring uncomfortable or unnatural postures. User feedback may reveal that a technically recognised gesture is difficult to perform repeatedly or that the selected vocabulary does not match real communication needs. These issues should inform later design iterations.

The project should be developed with a respectful understanding that sign languages are complete languages and are not simply collections of hand poses. A sensor glove can represent selected configurations and movements, but it does not automatically capture all linguistic features. Therefore, the most accurate description is a predefined gesture-to-text assistive prototype unless a broader sign-language vocabulary has been linguistically specified and validated.

B. Embedded Versus External Processing

Local inference can reduce the need to send sensor measurements to an external server and may improve availability when internet access is unreliable. It also introduces constraints on memory, processing time, model representation, and debugging. The chosen Random Forest export format is intended to make tree-based inference available on the microcontroller, but the export should be tested against the reference model. The local dashboard may help developers inspect values without becoming part of the core classification path.

If the vocabulary or model grows, the team may need to assess flash and RAM use, inference latency, and the effort required to update model files. These measurements are not available in the current evidence and should be collected if embedded efficiency is presented as a contribution. It is preferable to report a small number of repeatable measurements than to make broad claims about real-time performance without a defined measurement procedure.

VII. LIMITATIONS AND FUTURE WORK

The main limitation is the restricted gesture vocabulary and the absence of verified quantitative evaluation results in the available project evidence. The glove measures finger bending and inertial motion but does not capture facial expression, lip movement, full-body posture, or the spatial and grammatical structure of sign language. Its performance may also vary with hand size, glove fit, sensor placement, sensor fatigue, and changes in the electrical characteristics of the flex-sensor circuits.

The available labelled records cover baseline fingers-open and thumb-closed conditions; the other isolated-finger conditions and any additional gesture classes must be recorded from the physical device. No additional samples should be manufactured by assuming that a sensor value always increases when a finger bends. The observed direction and range must be established through measurement and calibration for each channel.

Future work should complete balanced data collection, record session and participant metadata, train and export a reproducible Random Forest model, and compare embedded predictions with the reference model. The model header and OLED detection issues reported by the diagnostic firmware should be resolved and retested. A controlled test protocol should then produce a confusion matrix and the standard classification metrics.

Additional improvements may include per-user calibration, sensor mounting refinements, drift monitoring, improved power management, low-battery indication, a larger but carefully defined vocabulary, and a configurable output mapping. Voice output may be considered as a later extension, but it should not be represented as part of the current completed system. Testing with intended users and accessibility specialists would help evaluate comfort, usefulness, and communication clarity.

A. Responsible Data Collection

Sensor datasets can contain information about a person's movement patterns. Data collection should be limited to what is needed for model development, and participants should understand how their samples will be stored and used. Identifying details should be separated from sensor files when possible. Any future study involving participants should follow the institution's applicable approval and consent requirements, particularly when recruiting people from the intended assistive-technology user group.

B. Roadmap for Validation

A practical next-step sequence is to complete isolated-finger data collection; verify sensor naming and feature order; train and evaluate the model with session-separated data; regenerate the embedded header; compare embedded predictions against a fixed reference set; repair and verify the OLED path; and run repeated complete-system trials. After these steps, the paper can be updated with a measured confusion matrix, classification metrics, and timing results. Additional participants and more varied recording sessions should be included before claiming generalisation.

VIII. CONCLUSION

This paper presents the design and implementation pathway for an ESP32-based smart gesture communication glove that combines five flex sensors with an MPU6050 IMU and a Random Forest-based gesture-to-text pipeline. The system is intended to recognise a selected vocabulary of predefined gestures and present corresponding text locally. Its camera-independent sensing arrangement offers a practical prototype direction for wearable assistive communication.

The system's effectiveness must be established through complete labelled data collection and reproducible testing. The available materials do not support numerical accuracy claims, and a subsequent firmware diagnostic session reported a missing model header and an OLED detection issue. The final submission should therefore distinguish verified implementation details from functionality that still requires validation. Completing the remaining gesture datasets, checking model export and display operation, and conducting a documented evaluation are the most important next steps.

REFERENCES

[1] L. Breiman, “Random Forests,” Machine Learning, vol. 45, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[2] Espressif Systems, “ESP32 Series Datasheet.” Consult the datasheet revision corresponding to the board used in the prototype.

[3] TDK InvenSense, “MPU-6000 and MPU-6050 Product Specification.” Consult the revision corresponding to the IMU module used.

[4] Solomon Systech Limited, “SSD1306: 128 × 64 Dot Matrix OLED/PLED Segment/Common Driver with Controller.” Retain this reference only if the fitted OLED module uses the SSD1306 controller.

[5] Spectra Symbol, “Flex Sensor” product datasheet or integration documentation. Confirm the exact sensor model used before final submission.